

Optimisation Numérique et Sciences des Données

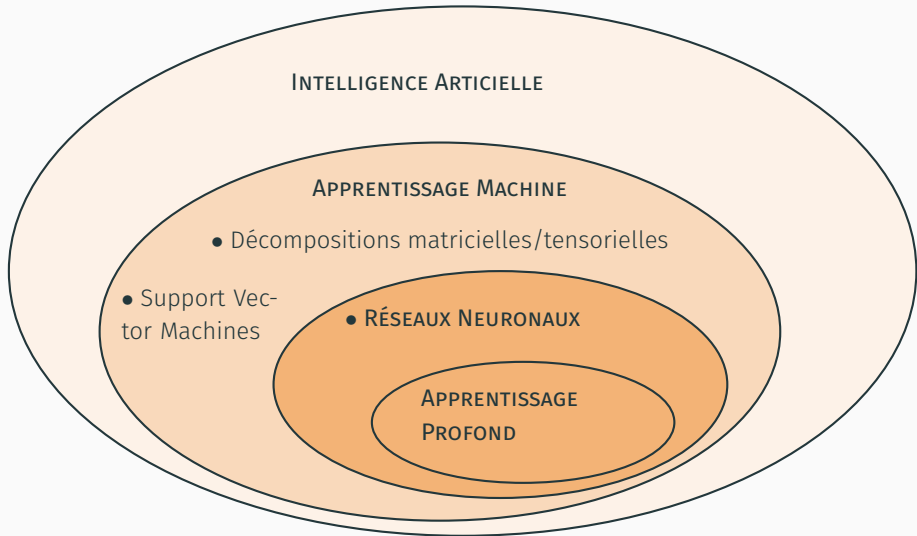
Sciences des données – Introduction aux réseaux de neurones

Paul Catala (CRAN, UL)
paul.catala(at)univ-lorraine.fr

Master Ingénierie des Systèmes Complexes
Semestre 9 – 2025

- <http://cedric-richard.fr/courses4.html>
- <https://speakerdeck.com/gpeyre/the-mathematics-of-neural-networks>
- <http://www.numerical-tours.com/>

1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Optimisation
5. Réseaux convolutifs



Apprentissage supervisé: **prédiction**

→ régression, classification, ...

Apprentissage non-supervisé: **extraction d'information**

→ segmentation, réduction de dimension, modèles génératifs, ...

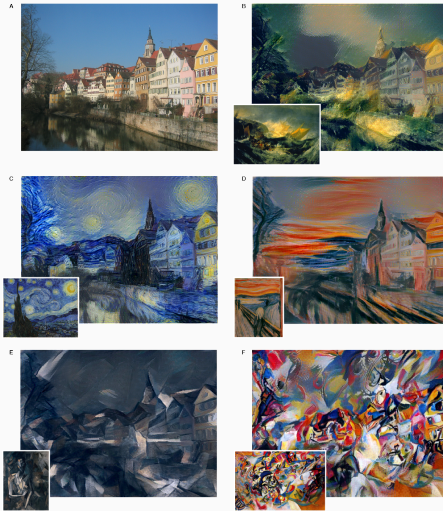
Apprentissage auto-supervisé

EXAMPLES – INPAINTING



Suvorov et al., *Resolution-robust Large Mask Inpainting* ..., 2021

EXEMPLES – TRANSFERT DE STYLE

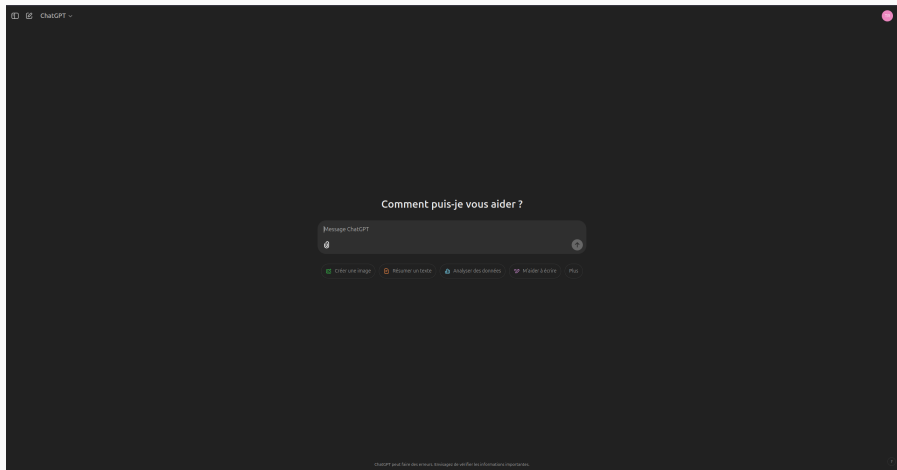


Gatys, Ecker et Bethge, *A Neural Algorithm of Artistic Style*, 2016

EXEMPLES – MODÈLE GÉNÉRATIF



Karras et al., *Progressive Growing of GANs for ...*, 2018



OpenAI, 2022

- Séparation de sources audio
- Transcriptions
- Segmentation audio

- Reconnaissance d'objet
- Recalage de forme
- Super-résolution

- Recommandations publicitaires
- Traduction
- Génération de sous-titres
- Génération de réponses

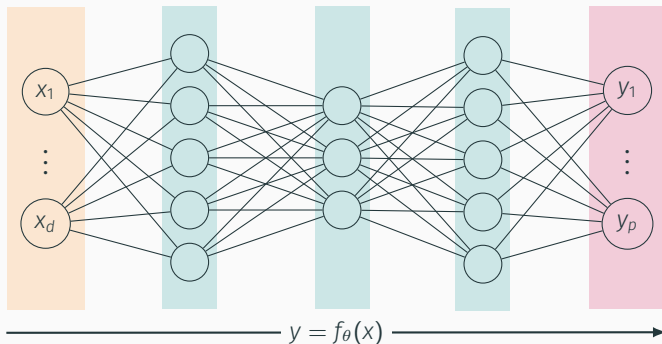
- etc...

- **Réseau de neurones** : ensemble de couches de "neurones" **apprenant** comment transmettre l'information d'une couche à l'autre à partir des données, afin de réaliser une tâche spécifique (classification, prédiction, traduction, génération, etc...)

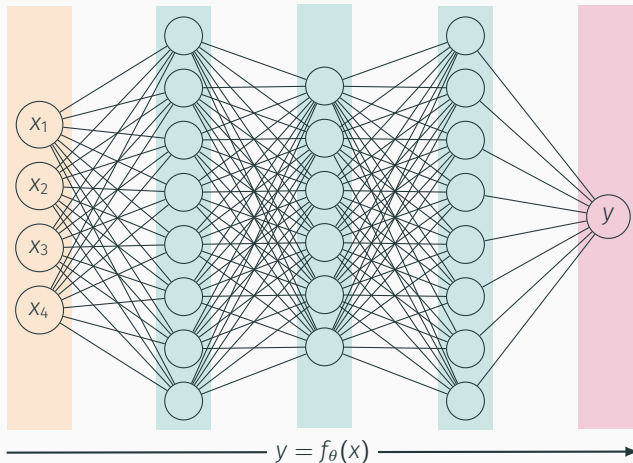
Couche d'entrée

Couches latentes (ou cachées)

Couche de sortie

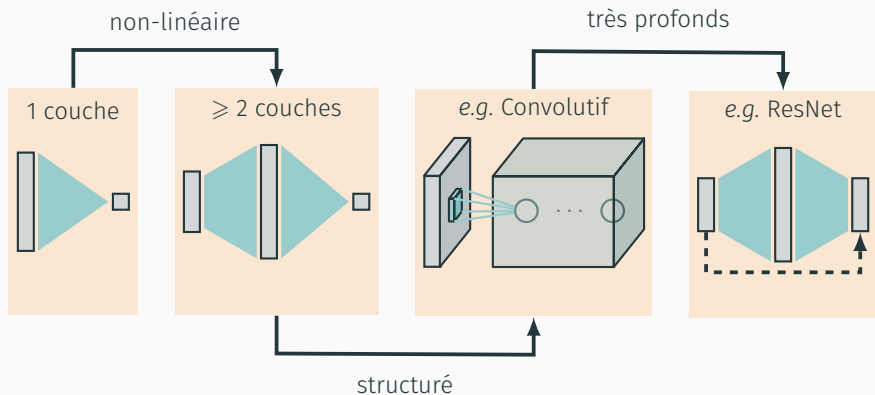


- Différentes tâches = différentes architectures de réseau, différents critères



Variables: pour de la classification, typiquement $\sim 10^6$ à 10^9 paramètres θ

HISTORIQUE



MLP, Rosenblatt, 1960s

CNN, e.g. Lecun, 1990s

↳ AlexNet, 2012

VAE, Welling, 2013

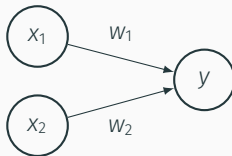
GAN, Goodfellow, 2014

ResNet, 2015

Transformers, 2017

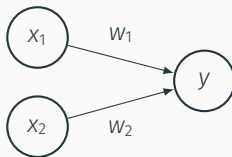
1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Optimisation
5. Réseaux convolutifs

- Un réseau de neurones est un **graphe de calcul**: les connections représentent des opérations mathématiques de base



- Sur cette représentation simple:
 - à chaque **connection** i est affecté un **poids** multiplicatif w_i
 - un neurone de sortie **additionne** les entrées

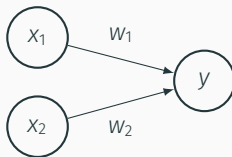
- Un réseau de neurones est un **graphe de calcul**: les connections représentent des opérations mathématiques de base



- Sur cette représentation simple:
 - à chaque **connection** i est affecté un **poids** multiplicatif w_i
 - un neurone de sortie **additionne** les entrées
- On note $\theta := (w_1, w_2)$ l'ensemble des paramètres du graphe. Ici on a représenté la fonction

$$y = f_{\theta}(x) =$$

- Un réseau de neurones est un **graphe de calcul**: les connections représentent des opérations mathématiques de base

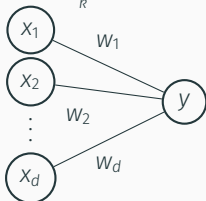


- Sur cette représentation simple:
 - à chaque **connection** i est affecté un **poids** multiplicatif w_i
 - un neurone de sortie **additionne** les entrées
- On note $\theta := (w_1, w_2)$ l'ensemble des paramètres du graphe. Ici on a représenté la fonction

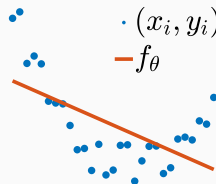
$$y = f_{\theta}(x) = w_1x_1 + w_2x_2$$

- Toute **fonction linéaire** peut se modéliser comme un réseau à une couche

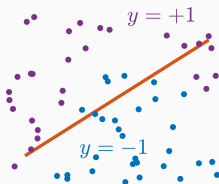
$$f_{\theta}(x) = \theta^{\top} x = \sum_k x_k w_k$$



Regression: $y = f_{\theta}(x)$



Classification: $f_{\theta}(x) = 0$



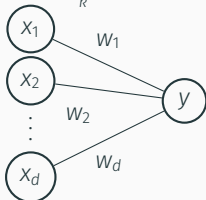
- Critère d'optimisation: à partir de **n données labellisées** $(x^{(i)}, y^{(i)}) \in \mathbb{R}^d \times \mathcal{Y}$

$$\min_{\theta \in \mathbb{R}^d} \sum_{i=1}^n \ell(\theta^{\top} x^{(i)}, y^{(i)})$$

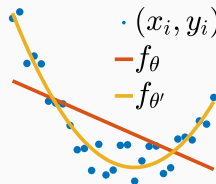
où ℓ est une fonction perte adéquate.

- Toute **fonction linéaire** peut se modéliser comme un réseau à une couche

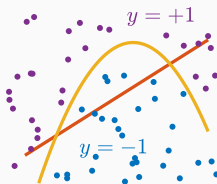
$$f_{\theta}(x) = \theta^{\top} x = \sum_k x_k w_k$$



Regression: $y = f_{\theta}(x)$



Classification: $f_{\theta}(x) = 0$



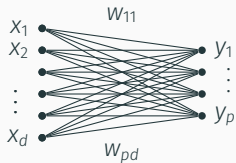
- Critère d'optimisation: à partir de **n données labellisées** $(x^{(i)}, y^{(i)}) \in \mathbb{R}^d \times \mathcal{Y}$

$$\min_{\theta \in \mathbb{R}^d} \sum_{i=1}^n \ell(\theta^{\top} x^{(i)}, y^{(i)})$$

où ℓ est une fonction perte adéquate.

- **Remarque:** $\theta \mapsto f_{\theta}(x)$ est linéaire, mais pas nécessairement $x \mapsto f_{\theta}(x)$:
 - **Méthodes à noyau:** remplacer x par $\varphi(x) \in \mathbb{R}^D$, avec $D \gg d$
 - **Apprentissage profond:** introduire de la non-linéarité pour apprendre φ

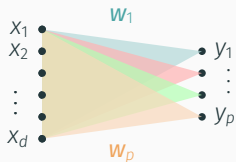
- Si la sortie est vectorielle



$$y_j = \sum_k w_{jk} x_k = \mathbf{w}_j^\top \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

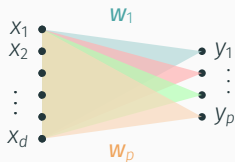
- Si la sortie est vectorielle



$$y_j = \sum_k w_{jk} x_k = \mathbf{w}_j^T \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_p^T \end{bmatrix}$$

- Si la sortie est vectorielle

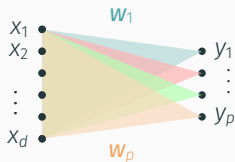


$$y_j = \sum_k w_{jk} x_k = \mathbf{w}_j^T \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_p^T \end{bmatrix}$$

- Si y est une **couche intermédiaire**, on affecte à chaque neurone y_j un **biais additif** b_j (non représenté graphiquement)

- Si la sortie est vectorielle



$$y_j = \sum_k w_{jk} x_k = \mathbf{w}_j^T \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots \\ w_{21} & w_{22} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_p^T \end{bmatrix}$$

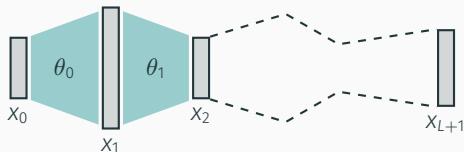
- Si $\mathbf{y}$ est une **couche intermédiaire**, on affecte à chaque neurone y_j un **biais additif** b_j (non représenté graphiquement)
- Dans ce cas, la couche modélise une fonction affine, *i.e.*

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}.$$

$\theta := (\mathbf{W}, \mathbf{b})$ sont les paramètres du réseau (pour cette couche).

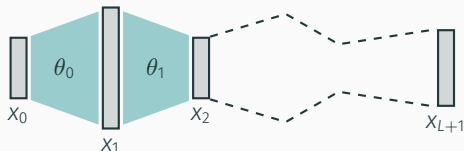
1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Optimisation
5. Réseaux convolutifs

- On ne gagne rien en rajoutant des couches de transmissions affines, puisque la composée de fonctions affines est une fonction affine



$$x_{L+1} = W_L x_L + b_L = \dots = (W_L \circ \dots \circ W_0) x_0 + \sum_{k=0}^L (W_L \circ \dots \circ W_{k+1}) b_k$$

- On ne gagne rien en rajoutant des couches de transmissions affines, puisque la composée de fonctions affines est une fonction affine



$$x_{L+1} = W_L x_L + b_L = \dots = (W_L \circ \dots \circ W_0) x_0 + \sum_{k=0}^L (W_L \circ \dots \circ W_{k+1}) b_k$$

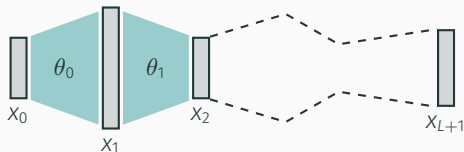
- Pour plus d'expressivité, on introduit des non-linéarités dans certaines couches

$$x_{k+1} := \sigma(W_k x_k + b_k)$$

où σ est une fonction non-linéaire.

Notation: $\sigma(\mathbf{z})$ s'entend par $\sigma(z_i)$ appliqué à chaque composante z_i de $\mathbf{z}$

- On ne gagne rien en rajoutant des couches de transmissions affines, puisque la composée de fonctions affines est une fonction affine



$$x_{L+1} = W_L x_L + b_L = \dots = (W_L \circ \dots \circ W_0) x_0 + \sum_{k=0}^L (W_L \circ \dots \circ W_{k+1}) b_k$$

- Pour plus d'expressivité, on introduit des non-linéarités dans certaines couches

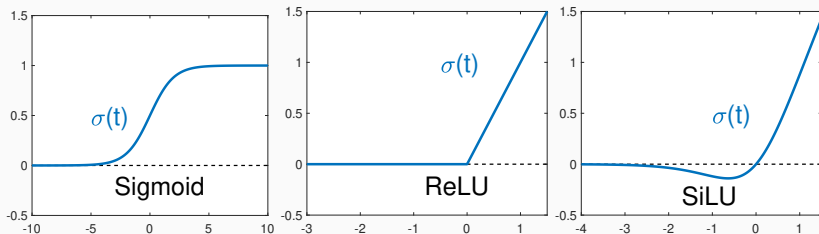
$$x_{k+1} := \sigma(W_k x_k + b_k)$$

où σ est une fonction non-linéaire.

Notation: $\sigma(\mathbf{z})$ s'entend par $\sigma(z_i)$ appliqué à chaque composante z_i de $\mathbf{z}$

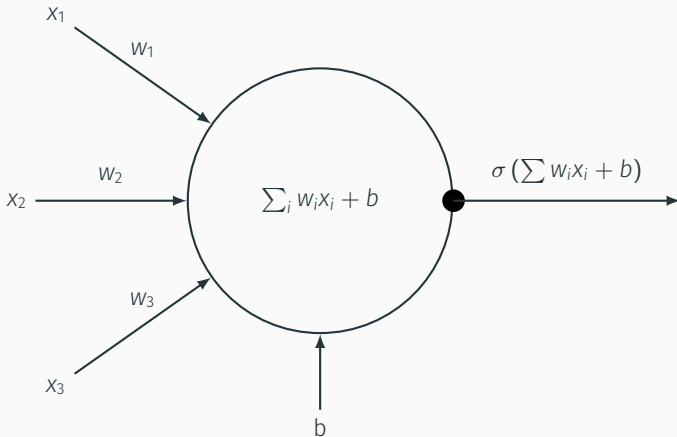
- Les réseaux résultants $(\theta_0, \theta_1, \dots, \theta_L)$ (i.e. entièrement connectés + non-linéarités) s'appellent des perceptrons.

- Pour les fonctions non-linéaires σ , on parle de **fonctions d'activation**.
- Pour une meilleure expressivité, on les choisit non-polynomiale

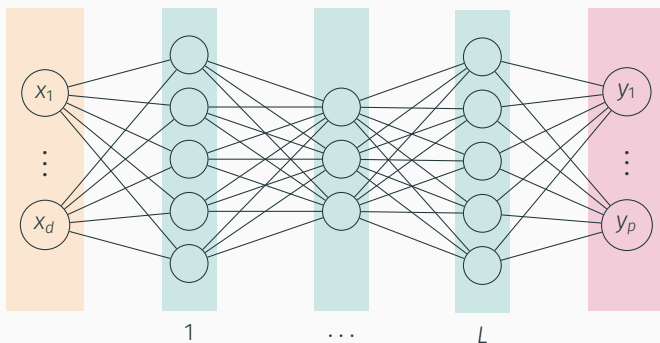


POUR RÉSUMER: NEURONE

Chaque neurone calcule une sortie à partir des entrées, via des opérations de **combinaisons linéaire** et d'**activation**.

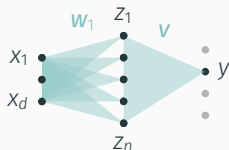


POUR RÉSUMER: RÉSEAU DE NEURONES



- La **couche d'entrée** ne compte pas
- Chaque **couche cachée** l est caractérisée par:
 - une matrice de poids $\mathbf{W}_{l-1}$ et un vecteur de biais $\mathbf{b}_{l-1}$
 - une fonction d'activation σ
- La **couche de sortie** est caractérisée par
 - une matrice de poids $\mathbf{W}_L$
- Le perceptron est entièrement défini par $\theta = ((\mathbf{W}_0, \mathbf{b}_0), \dots, (\mathbf{W}_{L-1}, \mathbf{b}_{L-1}), \mathbf{W}_L)$

THÉORÈME D'APPROXIMATION UNIVERSELLE



$$y = \sum_{k=1}^n v_k \sigma(\mathbf{w}_k^T \mathbf{x} + b_k)$$

- Un perceptron avec seulement une couche cachée (mais de **largeur arbitraire**) permet d'approcher n'importe quelle fonction continue (sur un compact)

Theorem 1 (Cybenko-Hornik). Soit $f : \mathcal{C} \mapsto \mathbb{R}$ une fonction continue. Alors pour tout $\varepsilon > 0$, il existe $n \in \mathbb{N}$ et $\theta = (\mathbf{W}, \mathbf{b}, \mathbf{v}) \in \mathbb{R}^{n \times d} \times \mathbb{R}^n \times \mathbb{R}^d$ tels que

$$\forall \mathbf{x} \in \mathcal{C}, \quad |f_{\theta}(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$$

1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Optimisation
5. Réseaux convolutifs

- **Entraînement** = optimisation des paramètres du réseau pour minimiser l'**erreur empirique** entre la sortie et les données (labellisées)

$$\theta_{\star} = \operatorname{argmin}_{\theta} J(\theta) := \frac{1}{n} \sum_{k=0}^n \ell(f_{\theta}(\mathbf{x}^{(k)}), \mathbf{y}^{(k)})$$

- Pas de solution analytique à ce problème: résolution par **descente de gradient**
- Calculer le gradient de J est compliqué: on a recours à la **différentiation automatique**, en particulier à l'algorithme de **rétropropagation du gradient** (Rumelhar, 1988).

- Le choix de la fonction perte $\ell(\hat{y}, y)$ ($\hat{y} = f_{\theta}(x)$) dépend de l'application visée.

$$\begin{array}{l|l} \text{MSE} & \frac{1}{p} \sum_{k=1}^p |\hat{y}_k - y_k|^2 \\ \text{MAE} & \frac{1}{p} \sum_{k=1}^p |\hat{y}_k - y_k| \end{array}$$

pour la régression

$$\begin{array}{l|l} \text{Hinge} & \frac{1}{p} \sum_{k=1}^p \max(0, 1 - \hat{y}_k y_k) \\ \text{Log} & \frac{1}{p} \sum (y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)) \end{array}$$

pour la classification

- **Descente de gradient**: suit la direction de plus grande pente en chaque point

$$\theta_{t+1} = \theta_t - \frac{\alpha}{n} \sum_{k=1}^n \nabla_{\theta}(\ell(f_{\theta_t}(x^{(k)}), y^{(k)}))$$

Le pas α est souvent appelé **learning rate**.

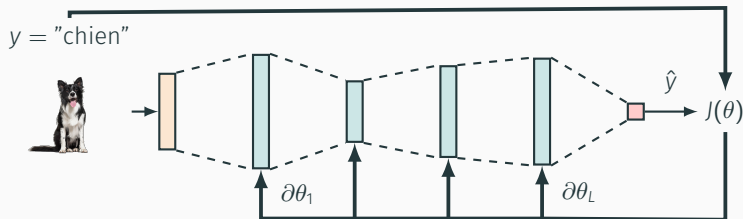
- **Descente de gradient stochastique**: approche le gradient à partir d'une donnée d'apprentissage.

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta}(\ell(f_{\theta_t}(x^{(k_t)}), y^{(k_t)}))$$

- **Descente de gradient stochastique mini-batch**: approche le gradient à partir d'un sous-ensemble $\mathcal{B}_t$ de données d'apprentissage.

$$\theta_{t+1} = \theta_t - \frac{\alpha}{|\mathcal{B}_t|} \sum_{k \in \mathcal{B}_t} \nabla_{\theta}(\ell(f_{\theta_t}(x^{(k)}), y^{(k)}))$$

$$J(\theta) := \frac{1}{n} \sum_{k=0}^n \ell(f_{\theta}(x^{(k)}), y^{(k)})$$

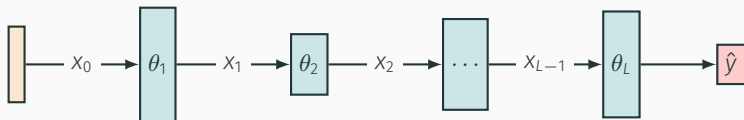


$$\hat{y} = f_L \circ f_{L-1} \circ \dots \circ f_1(x) = f_{\theta}(x) = f(x; \theta_1, \dots, \theta_L)$$

- La minimisation de $J(\theta)$ nécessite de pouvoir dériver la sortie du réseau par rapport aux paramètres $\theta = (\theta_1, \dots, \theta_L)$ du réseau.
- Autrement dit, on doit calculer

$$\nabla J = \left(\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_L} \right)^T$$

- Pour estimer ce gradient, on exploite le principe de la **rétropropagation du gradient** ("backpropagation")



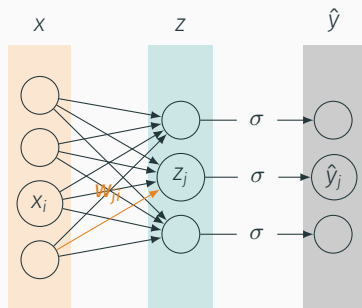
- Propagation avant dans le réseau:

$$\hat{y} = f_{\theta_L} \circ f_{\theta_{L-1}} \circ \dots \circ f_1(x) = f_{\theta}(x)$$

- Propagation arrière: on introduit les étapes intermédiaires x_l

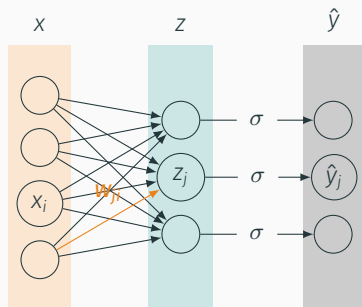
$$\hat{y} = f_{\theta_L} \circ \dots \circ f_{\theta_{l+1}}(x_l)$$

- On calcule le gradient à rebours $\partial\theta_L \rightarrow \partial\theta_{L-1} \rightarrow \dots$



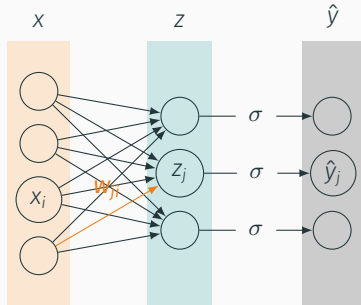
$$\hat{y} = \sigma(Wx + b), \quad z = Wx + b, \quad F(\theta) := \ell(\hat{y}(\theta), y)$$

"Chain rule" $\left\{ \frac{\partial F}{\partial w_{ji}} \right\}$



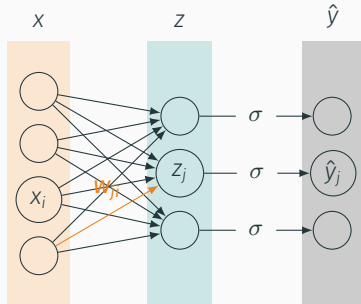
$$\hat{y} = \sigma(Wx + b), \quad z = Wx + b, \quad F(\theta) := \ell(\hat{y}(\theta), y)$$

"Chain rule"
$$\left\{ \frac{\partial F}{\partial w_{ji}} = \frac{\partial F}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} \right.$$



$$\hat{y} = \sigma(Wx + b), \quad z = Wx + b, \quad F(\theta) := \ell(\hat{y}(\theta), y)$$

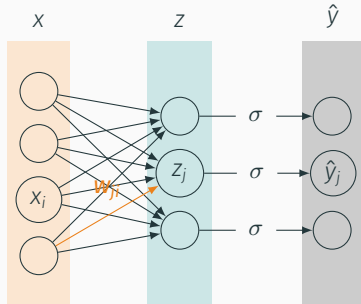
"Chain rule" $\left\{ \begin{array}{l} \frac{\partial F}{\partial w_{ji}} = \frac{\partial F}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} = \frac{\partial \ell}{\partial \hat{y}_j} \cdot \sigma'(z_j) \cdot x_i \end{array} \right.$



$$\hat{y} = \sigma(Wx + b), \quad z = Wx + b, \quad F(\theta) := \ell(\hat{y}(\theta), y)$$

"Chain rule"

$$\begin{cases} \frac{\partial F}{\partial w_{ji}} = \frac{\partial F}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} = \frac{\partial \ell}{\partial \hat{y}_j} \cdot \sigma'(z_j) \cdot x_i \\ \frac{\partial J}{\partial b_j} = \frac{\partial \ell}{\partial \hat{y}_j} \cdot \sigma'(z_j) \end{cases}$$



$$\hat{y} = \sigma(Wx + b), \quad z = Wx + b, \quad F(\theta) := \ell(\hat{y}(\theta), y)$$

"Chain rule"
$$\begin{cases} \frac{\partial F}{\partial w_{ji}} = \frac{\partial F}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} = \frac{\partial \ell}{\partial \hat{y}_j} \cdot \sigma'(z_j) \cdot x_i \\ \frac{\partial J}{\partial b_j} = \frac{\partial \ell}{\partial \hat{y}_j} \cdot \sigma'(z_j) \end{cases}$$

$$\nabla_w F(W, b) = [\sigma'(z) \odot \nabla \ell(\hat{y}, y)] \cdot x^\top$$

$$\nabla_b F(W, b) = \sigma'(z) \odot \nabla \ell(\hat{y}, y)$$

- Calcul du gradient de F par rapport à θ : une propagation avant, puis une rétropropagation

- $x_0 \leftarrow x$
- Pour l de 0 à $L - 1$:
 - $z_{l+1} \leftarrow W_l x_l + b_l$
 - $x_{l+1} \leftarrow \sigma_l(z_l)$
- $\hat{y} \leftarrow x_L$

Propagation

- $g \leftarrow \nabla \ell(\hat{y}, y)$
- Pour l de L à 1:
 - $g \leftarrow \sigma'_l(z_l) \odot g$
 - $\nabla_w F \leftarrow g x_l^\top$
 - $g \leftarrow W_l^\top g$
- Renvoyer $\nabla_w J$

Rétropropagation

- Plus généralement: différentiation automatique

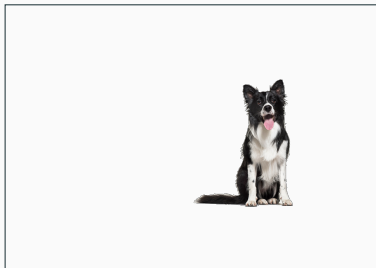
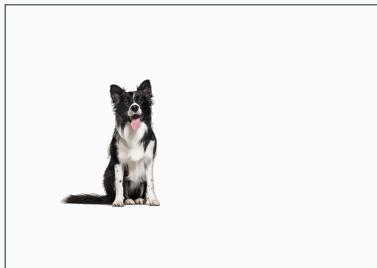
- Les multiplications de gradient successives lors de la propagation à rebours ont tendance à tendre vers 0
- **Vanishing gradient**: problèmes de stabilité
- Les **skip connections** introduites avec l'architecture ResNet (He, Zhang, Ren, Sun, 2015) ont révolutionné l'apprentissage des réseaux très profonds (100+ couches)
- Idée: rajouter des connections directes $y = f(x) + x$ entre des couches potentiellement éloignées.

1. Introduction
2. Modèle linéaire
3. Réseaux multi-couches
4. Optimisation
5. Réseaux convolutifs

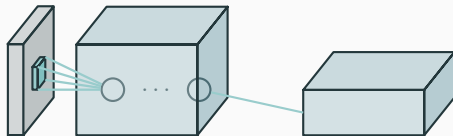
- Pour une image $256 \times 256 \times 3$ pixels, le nombre de poids à optimiser pour un neurone de la première couche d'un MLP est de l'ordre de $2 \cdot 10^5 \rightarrow$ trop coûteux

- Pour une image $256 \times 256 \times 3$ pixels, le nombre de poids à optimiser pour un neurone de la première couche d'un MLP est de l'ordre de $2 \cdot 10^5 \rightarrow$ trop coûteux
- Les MLP sont des réseaux entièrement connectés: aucun *a priori* de structure, alors que les données sont toujours des propriétés structurelles fortes

- Pour une image $256 \times 256 \times 3$ pixels, le nombre de poids à optimiser pour un neurone de la première couche d'un MLP est de l'ordre de $2.10^5 \rightarrow$ trop coûteux
- Les MLP sont des réseaux entièrement connectés: aucun *a priori* de structure, alors que les données sont toujours des propriétés structurelles fortes
- Pour des images par exemple: corrélation spatiale, *invariance par translation*, ...



CONVOLUTION NEURAL NETWORKS (CNN)



- Chaque neurone n'est connecté qu'à un petit nombre de neurones
- Un même poids s'applique à un groupe de neurones (masque de convolution)
- Opération de convolution: favorise l'invariance par translation
- Opération de pooling: réduit la dimension

Un CNN est un empilement (en alternant) de

- couches de convolution
- couches d'activation
- couches de pooling
- couches entièrement connectées

CONVOLUTION

0	0	0	0	0
0	193	187	200	...
0	185	163	160	...
0	185	160	161	...
0	190	189	170	...
0	193	187	171	...
0	201	180	184	...
...	...	...	...	...

0	0	0	0	0
0	168	162	163	...
0	160	161	164	...
0	156	158	162	...
0	155	155	158	...
0	154	152	157	...
0	153	152	153	...
...	...	...	...	...

0	0	0	0	0
0	167	166	167	...
0	164	165	168	...
0	160	162	168	...
0	156	156	159	...
0	155	158	158	...
0	157	156	159	...
...	...	...	...	...

-1	1	-1
1	-1	1
-1	1	-1

16

+

0	1	0
1	1	1
0	1	0

490

+

-1	1	0
0	-1	0
1	-1	1

-166

+ bias

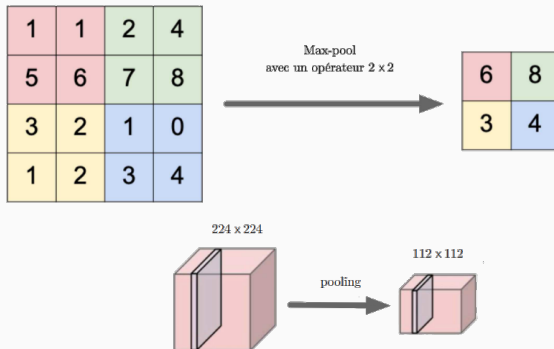
zero-padding

=

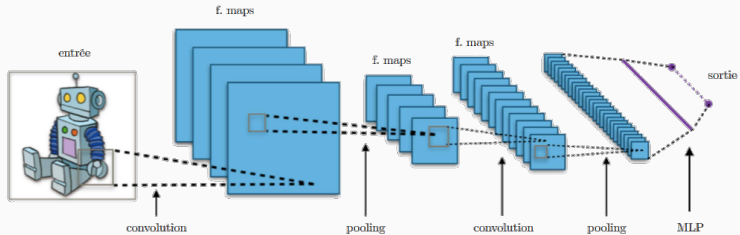
349	...	...
...	...	...
...	...	...

Activation map

POOLING



<http://cedric-richard.fr/assets/files/ML-CO-NN.pdf>



<http://cedric-richard.fr/assets/files/ML-CO-NN.pdf>

[illegible]

